

ECE391
Computer System Engineering
Lecture 6

University of Illinois at Urbana- Champaign

Fall 2024

Lecture Topics

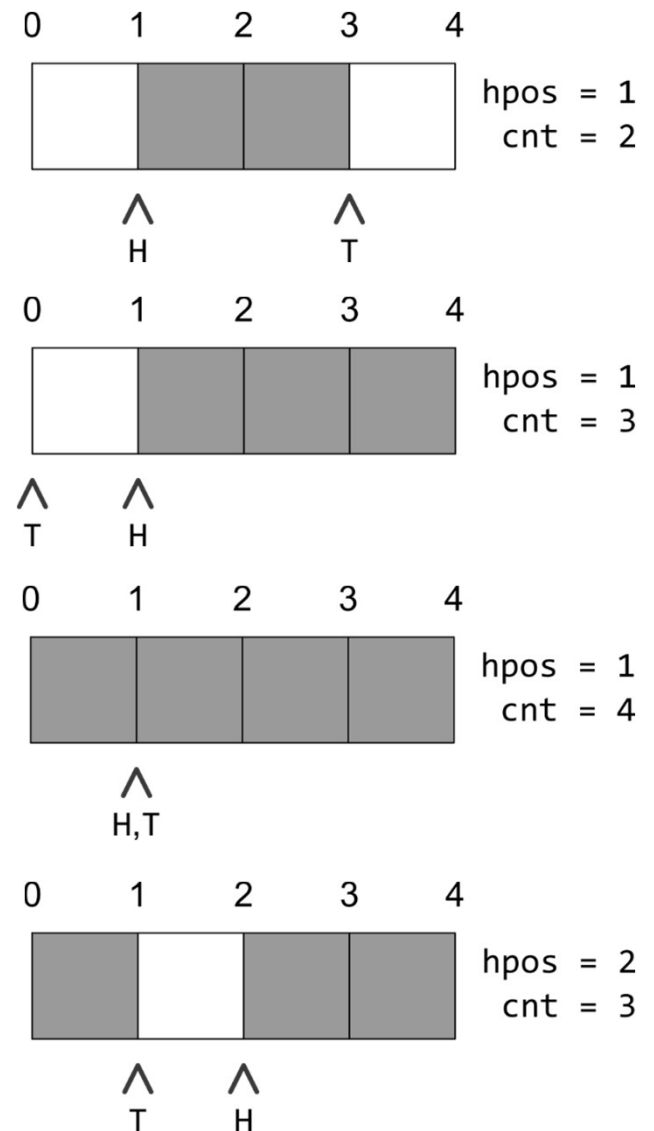
- Shared resources
- Critical sections

Interrupt-Based I/O

- Let's implement the serial port driver using interrupts
- We can ask the NS8250 to raise an interrupt when:
 - DR = 1: RBR contains a received byte
 - THRE = 1: transmitter holding register ready for next byte
- We will read the received bytes as soon as they are available, so we don't miss any
- We'll use a ring buffer data structure

Ring Buffer

- A ring buffer is a fixed-size FIFO
- hpos (head position):
index of next element to be removed
- tpos (tail position):
index where next element will be inserted
- cnt (count):
number of elements currently in buffer
- $tpos = (hpos + cnt) \% size$
so only need to store two of the three variables above*



```
static char rxbuf[4];
static int rxhpos = 0;
static int rxcnt = 0;

// rxdata called from interrupt handler when
// a byte is received and available in RBR
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;

}

char getchar(void) {

}

}
```

```

static char rxbuf[4];
static int rxhpos = 0;
static int rxcnt = 0;

// getchar() is executing ...

char c;

while (rxcnt == 0) {
    // wait for data to become available
}

c = rxbuf[rxhpos];
rxhpos = (rxhpos + 1) % 4;

// interrupt fires, we're in rxdata()

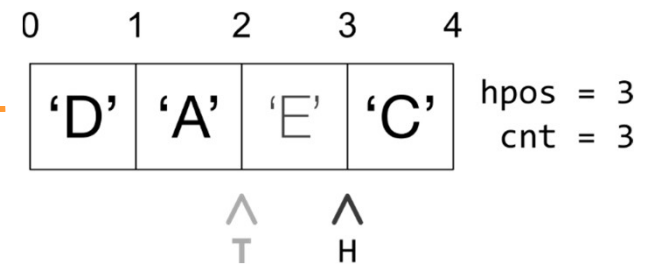
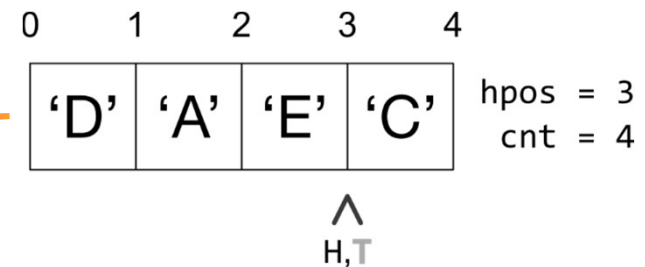
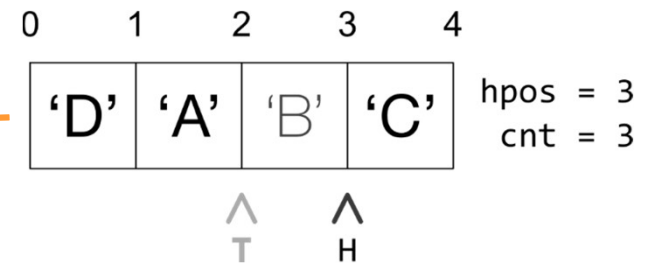
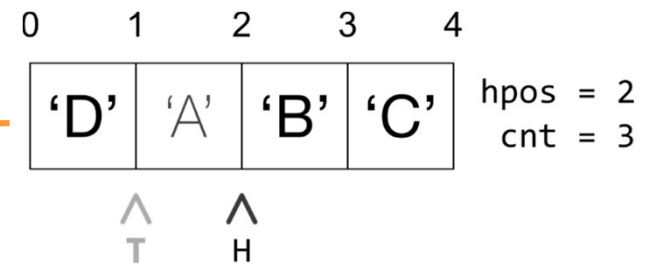
int tpos = (rxhpos + rxcnt) % 4;

if (rxcnt < 4) {
    // read byte from RBR
    rxbuf[tpos] = inb(0x3F8);
    rxcnt = rxcnt + 1;
} else {
    // receiver buffer overflow
}

// back to getchar()
rxcnt = rxcnt - 1;

return c;

```



Invariants

- Most non-trivial data structures have **invariants** — properties that must hold *before* and *after* operations on the data structure
- Invariant may be temporarily invalid *during* operation
- What is an invariant of the ring buffer?
- `cnt` is number of elements in FIFO/queue
- $(\text{hpos}+0) \% \text{size}$ is index of oldest element,
 $(\text{hpos}+1) \% \text{size}$ is index of next oldest, *etc.*
- Interleaved execution of operations breaks invariant

You are now Entering the Atomic Age

- Ring buffer ops should be **atomic**:

```
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}

char getchar(void) {
    char c;
    while (rxcnt == 0) {
        // wait for data to become available
    }
    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```


Disabling Interrupts in Linux

- `local_irq_disable()` disables interrupts (IF = 0)
- `local_irq_enable()` enables interrupts (IF = 1)
- `local_irq_save(flags)` saves EFLAGS register in flags variable and then disables interrupts (`cli`)
 - This is a macro, flags passed by name, not value
- `local_irq_restore(flags)` restores EFLAGS

Other Kinds of Critical Sections

- Updating a data structure in memory is one place where we need critical sections
- May also need critical sections when dealing with some devices
 - Devices have state
 - Processors interact with devices using specific protocols
 - Protocol often requires several steps (e.g., I/O instructions)
 - Device cannot tell which piece of code did an operation

Example: Divisor Latch Register on NS8250

- Not all NS8250 registers are always accessible
- To write Divisor Latch register need to set DLAB = 1
 - DLAB is bit 7 of the Line Control Register (LCR) at 0x3F8+3
- When DLAB = 0
 - Byte at 0x3F8+0 is Receive Buffer Register (RBR)
 - Byte at 0x3F8+1 is Interrupt Enable Register (IER)
- When DLAB = 1
 - Word at 0x3F8+0 is 16-bit Divisor Latch register
- Same port address (0x3F8+0) maps to different registers depending on DLAB bit!

C Code for Divisor Latch Update on NS8250

```
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}
```

```
void setdiv(uint16_t n) {
    uint8_t lcr;
    // Set DLAB=1 in LCR (0x3F8+3)
    lcr = inb(0x3F8+3);
    outb(0x3F8+3, lcr | (1 << 7));
    // Write Divisor Latch register
    outw(0x3F8+0, n);
    // Restore previous value of LCR
    outb(0x3F8+3, lcr & ~(1 << 7));
}
```

C Code for Divisor Latch Update on NS8250

```
void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}
```

```
void setdiv(uint16_t n) {
    unsigned long flags;
    uint8_t lcr;
    local_irq_save(flags);
    // Set DLAB=1 in LCR (0x3F8+3)
    lcr = inb(0x3F8+3);
    outb(0x3F8+3, lcr | (1 << 7));
    // Write Divisor Latch register
    outw(0x3F8+0, n);
    // Restore previous value of LCR
    outb(0x3F8+3, lcr & ~(1 << 7));
    local_irq_restore(flags);
}
```

Concurrent Execution

- Two threads or pieces of code are said to execute **concurrently**
- Two threads or pieces of code are said to execute **in parallel** if
- Parallel implies concurrent, but not vice versa
- Dealing with concurrency requires special mechanisms to ensure program correctness

Interrupts Inside Interrupts

- Recall that CPU disables interrupts on entry to an interrupt handler
 - Handler restores IF when it returns (`iret` instr.)
- An interrupt handler may re-enable interrupts before it finishes to allow other interrupts to be serviced
 - Need to consider possibility of nested interrupt handler invocation ... including the same interrupt handler being called again before it returns
- Some functions may be called from normal and interrupt handler context
 - Need to be very careful

```

static char rxbuf[4];
static int rxhpos = 0;
static int rxcnt = 0;

// rxdata called from interrupt handler when
// a byte is received and available in RBR

void rxdata(void) {
    int tpos = (rxhpos + rxcnt) % 4;
    if (rxcnt < 4) {
        // read byte from RBR
        rxbuf[tpos] = inb(0x3F8);
        rxcnt = rxcnt + 1;
    } else {
        // receiver buffer overflow
    }
}

char getchar(void) {
    char c;

    while (rxcnt == 0) {
        // wait for data to become available
    }

    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;

    return c;
}

```


Compiler Optimization

- `getchar()` *without* compiler optimization:

```
_getchar:
    pushl    %ebp
    movl     %esp, %ebp
    subl     $8, %esp

LBB0_1:
    cmpl     $0, _rxcnt
    jne      LBB0_3
    jmp      LBB0_1

LBB0_3:
    movl     _rxhpos, %ecx
    movb     _rxbuf(,%ecx), %dl
    ...
    ret
```

```
static char rxbuf[4];
static int rxhpos = 0;
static int rxcnt = 0;

char getchar(void) {
    char c;
    while (rxcnt == 0)
        continue;

    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```

Compiler Optimization

- `getchar()` *with* compiler optimization:

```
_getchar:
    pushl    %ebp
    movl     %esp, %ebp
LBB0_1:
    jmp      LBB0_1
```

```
static char rxbuf[4];
static int rxhpos = 0;
static int rxcnt = 0;

char getchar(void) {
    char c;
    while (rxcnt == 0)
        continue;

    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```

Volatile Keyword

- Use the `volatile` C keyword to tell the compiler that the value of a variable in memory may change and access has side-effects
 - Forces the compiler to reload the variable on each access
- Use `volatile` for variables that may change outside compiler control
 - Variables that may be changed in an interrupt handler
 - Variables that may be changed in another thread
 - Preemptable thread execution
 - Multiprocessor systems
 - Memory-mapped I/O
 - Variable is a device register (access via pointer to `volatile`)
 - DMA-modified memory

Compiler Optimization

- `getchar()` *with* compiler optimization:

```
_getchar:
    pushl    %ebp
    movl     %esp, %ebp
LBB0_1:
    cmpl     $0, _rxcnt
    je       LBB0_1
    movl     _rxhpos, %eax
    movsbl   _rxbuf(%eax), %eax
```

```
static volatile char rxbuf[4];
static volatile int rxhpos = 0;
static volatile int rxcnt = 0;

char getchar(void) {
    char c;
    while (rxcnt == 0)
        continue;

    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```

What Volatile is Not

- Volatile keyword only affects its variable

```
_getchar:
    pushl    %ebp
    movl     %esp, %ebp
LBB0_1:
    cmpl     $0, _rxcnt
    je       LBB0_1
    movl     _rxhpos, %eax
    ...
    xorl     %eax, %eax
    popl     %ebp
    ret
```

```
static char rxbuf[4];
static volatile int rxhpos = 0;
static volatile int rxcnt = 0;

char getchar(void) {
    char c;
    while (rxcnt == 0)
        continue;
    c = rxbuf[rxhpos];
    rxhpos = (rxhpos + 1) % 4;
    rxcnt = rxcnt - 1;
    return c;
}
```

Summary So Far

- Two contexts: interrupt handlers and main code
- Interrupt handlers may interrupt execution of other code at any time when interrupts enabled ($IF = 1$)
- Disable interrupts to prevent interrupt handlers from running while we're in a critical section
- *Unless explicitly re-enabled*, interrupt handlers run with interrupts disabled
- All of this applies to single-processor systems only
- Will discuss multiple CPUs next week